

1. Execution Context (EC)

The environment where JS code runs. Two types exist:

- **Global EC** — created first, lives at the bottom of the call stack.
- **Function EC** — created on every function call, destroyed on return.

Every EC goes through **two phases**:

Phase	What happens	Key detail
Phase 1 Memory Creation	JS scans all code before running anything.	var → undefined Functions → full code stored let/const → TDZ
Phase 2 Code Execution	Runs line by line, assigns actual values.	Variables get real values Function calls trigger new ECs

2. Execution Context — Full Walkthrough

```
let n = 2;
function square(num) {
  let ans = num * num;
  return ans;
}
let square2 = square(n);
let square4 = square(square2);
```

Global EC — Phase 1 (Memory Creation):

Variable	Value allocated
n	undefined
square	full function code stored
square2	undefined
square4	undefined

Global EC — Phase 2 (Code Execution):

- n → assigned 2
- square(n) called → **new Function EC created & pushed onto call stack**

Function EC for square(2):

Variable	Phase 1	Phase 2
num	undefined	assigned 2
ans	undefined	2 × 2 = 4 → returned

- Returns 4 → EC destroyed → square2 = 4

Function EC for square(square2) i.e. square(4):

- Same two-phase process repeats — num = 4, ans = 16
- Returns **16** → EC destroyed → square4 = 16
- Global EC finishes → also destroyed.

3. Call Stack — In Depth

A **LIFO (Last In, First Out)** data structure that tracks which EC is currently running.

```
Start      → [ Global EC ]
square(2)  → [ square EC ] [ Global EC ]
return 4   → [ Global EC ]      ← square EC popped
square(4)  → [ square EC ] [ Global EC ]
return 16  → [ Global EC ]      ← square EC popped
Program ends → []              ← Global EC popped
```

Key rules:

- GEC is always pushed **first** and popped **last**
- A new EC is pushed **every time** a function is called
- After **return**, that EC is immediately popped and destroyed
- Whatever EC is on **top** is what is currently executing
- Nested function calls stack on top of each other

Other names: Execution Context Stack · Program Stack · Control Stack · Runtime Stack · Machine Stack

4. Hoisting — With Examples

Hoisting is **Phase 1 memory allocation** happening before any code runs. It is **not** literal code movement — the engine just prepares memory early.

Function Declaration — fully hoisted:

```
greet(); // works — prints "Hello"
function greet() { console.log("Hello"); }
```

var — hoisted as undefined:

```
console.log(x); // undefined (not an error)
var x = 5;
console.log(x); // 5
```

let / const — Temporal Dead Zone (TDZ):

```
console.log(y); // ReferenceError
let y = 10;
```

let/const are technically hoisted but kept in TDZ — completely inaccessible until the declaration line.

Arrow Function / Function Expression — treated like a variable:

```
console.log(fn); // undefined
fn();             // TypeError: fn is not a function
var fn = () => console.log("Hi");
fn();             // works fine after assignment
```

Hoisting Quick Reference:

Type	Phase 1 value	Before declaration	After declaration
Function declaration	Full code	Works fine	Works
var	undefined	Returns undefined	Works
let / const	TDZ	ReferenceError	Works
Arrow fn / fn expr	undefined	TypeError	Works

undefined = variable exists but no value yet. **not defined** = variable never declared, no memory allocated at all.

5. Variable Environment & Shadowing

```
var x = 1;
function a() { var x = 10; console.log(x); } // 10
function b() { var x = 100; console.log(x); } // 100
a(); // 10
b(); // 100
console.log(x); // 1
```

Each function call creates a **completely isolated memory space**. Even though all three use the name x, they never interfere.

Active EC	Local x value	console.log(x) prints
Global	x = 1	1
a() running	x = 10	10
b() running	x = 100	100
Back to Global	x = 1	1

Variable Shadowing:

- When a local variable has the same name as a global, the local version **shadows** (hides) the global inside that function.
- The global is **untouched**.

JS lookup order: current local EC → outer scope → global. This is the foundation of **scope chain** and later **closures**.

6. Global Object & Window

When JS starts in a browser, even before your code runs, the engine creates a **Global Execution Context** and a **global object** — in browsers this is **window**.

```
var name = "Akash";
console.log(name); // "Akash"
console.log(window.name); // "Akash" - same thing

function greet() { console.log("Hi"); }
window.greet(); // works exactly the same
```

Key behaviours:

- window** is the global object only in browsers (Node.js uses **global**)

- **this** at the global level also refers to window
- **var** at global scope → becomes window.x
- **let / const** at global scope → does NOT attach to window
- Avoid polluting the global namespace — use functions or modules.

```
// Bad – pollutes window
var count = 0;

// Good – encapsulated
function init() { let count = 0; }
```

Master Recap

Execution Context	Environment JS code runs in — always 2 phases
Memory Phase	Everything gets undefined or fn code before execution
Code Phase	Line-by-line, actual values assigned, functions called
Call Stack	LIFO stack — tracks which EC is running right now
Hoisting	Phase 1 memory prep; fn declarations fully ready, vars get undefined
TDZ	let/const exist in memory but inaccessible until declaration
Variable Env	Each function has isolated memory — no leakage between scopes
Shadowing	Local var hides global var of same name inside that function
Window / Global	Browser's global scope container — all global vars live on window